

CODTECH IT SOLUTIONS PRIVATE LIMITED**MERN Stack Web Development Internship — Project Report****MINI PROJECT REPORT****TOPIC: MERN E-Commerce Storefront – Frontend Application**

Candidate Name	Yug Thakral
Intern ID	CITS3144
Organization	Codtech IT Solutions Private Limited
Internship Period	03 June 2026 – 15 July 2026
Duration	6 Weeks
Domain	MERN Stack Web Development
Project Type	Full-Stack E-Commerce Web Application (Frontend)
Tools / Language	React.js, Material UI, Axios, React Router DOM, Context API, Recharts

Supervised by

Neela Santhosh Kumar

Human Resources & Academic Head

Codtech IT Solutions Private Limited

www.codtechitsolutions.com | Hr@codtechitsolutions.com | +91 9848925128

8-7-7/2, Plot No.51, Opp: Naveena School, Hashthinapuram Central, Hyderabad, 500 079, Telangana

1. Problem Statement and Background

Online shopping has become one of the fastest-growing industries as internet access and digital payment adoption continue to rise. Customers today expect fast product browsing, secure transactions, and a personalized shopping experience, while store administrators need a simple way to manage inventory, users, and orders. Traditional storefront frontends often fall short of these expectations: they tend to have static, page-reloading interfaces, poor mobile responsiveness, limited component reuse, and cumbersome admin tooling.

This project addresses that gap by building a modern, responsive React frontend for a MERN-stack e-commerce platform, covering the complete customer journey — browsing, cart and wishlist management, checkout, and order tracking — alongside a dedicated administrator dashboard for managing products, users, and orders.

2. Project Overview

The MERN E-Commerce Frontend is a responsive, component-based web application built with React.js. It serves as the client-side interface of a complete MERN stack e-commerce platform, communicating with a Node.js and Express REST API backed by MongoDB. Customers can register, log in, browse and search products, manage a shopping cart and wishlist, place orders, and track order history. Administrators are provided with a separate dashboard for managing products, users, and orders, including interactive sales and revenue charts built with Recharts.

The application emphasizes performance, clean UI, modular architecture, and centralized state management using the React Context API, avoiding prop drilling and keeping the codebase maintainable as it grows.

3. Project Objectives

- Build a responsive, single-page shopping website using React.
- Provide secure, JWT-based user authentication with registration, login, and password recovery.
- Implement reusable, component-based UI elements across customer and admin views.
- Consume backend REST APIs efficiently using Axios.
- Build an interactive admin dashboard with statistics and data visualization.
- Maintain a scalable, well-organized folder architecture.
- Improve overall user experience through Material UI components and consistent styling.

4. Technology Stack

4.1 Frontend Technologies

React.js drives the single-page application, rendering the navbar, product listings, product detail pages, cart, wishlist, checkout flow, and the admin dashboard as reusable components. React Router DOM manages client-side routing across public, authenticated, and admin routes without full page reloads.

Material UI and styled-components provide a consistent, responsive visual language, while the React Context API centralizes state for the logged-in user, authentication status, cart, wishlist, and orders — removing the need for deep prop drilling. Axios handles all REST API communication with the backend, React Toastify

surfaces success, error, and warning notifications, React Icons supplies iconography, and Recharts powers the admin dashboard's sales and revenue charts.

4.2 Backend and Supporting Technologies

The frontend is designed to work against a Node.js and Express.js REST API, with MongoDB as the persistence layer for products, users, and orders. Development tooling includes VS Code, Git and GitHub for version control, Postman for API testing, and npm for package management.

5. System Architecture

The application follows a straightforward client–server architecture: the React single-page application in the browser makes REST calls through Axios to an Express and Node.js backend, which in turn reads from and writes to a MongoDB database.

- User → React Frontend (SPA)
- React Frontend → Axios REST API calls
- Express + Node backend → processes requests, applies business logic and auth checks
- Backend → MongoDB database for persistence

6. Features

6.1 Customer-Facing Features

- Authentication — registration, login, JWT-based sessions, forgot/reset password.
- Product browsing — product listing, search, detail pages, and reviews.
- Cart — add/remove items, adjust quantity, and automatic total calculation.
- Wishlist — add, remove, and view saved products.
- Orders — place an order, view order history, and track order status.
- Payments — integration point for a secure payment gateway.
- Notifications — success, error, and warning toasts throughout the app.

6.2 Administrator Features

- Product management — add, update, delete, and view product details.
- User management — view users, add users, and inspect user details.
- Order management — view orders, update order status, and track customer orders.
- Dashboard — total products, total orders, total users, sales statistics, and revenue charts rendered with Recharts.

7. Folder Structure

The codebase is organized under `src/` into clearly separated areas: an `Admin/` module (with its own `Auth`, `Components` — including `Charts`, `Tables`, and `UserData` — and `Pages`), a shared `Assets/` directory, an `Auth/` module (`Login`, `Register`, `ForgotPassword`), reusable `Components/`, a `Context/` folder for global state, customer-

facing Pages/, a Routes/ configuration folder, and the root App.js and index.js entry points. This separation keeps customer and admin concerns independent while sharing common building blocks.

8. React Component Architecture

The application follows a component-based architecture rooted at App, which composes the Navbar, Home, ProductCard, ProductDetails, Cart, Wishlist, Footer, and the Admin Dashboard as independent, reusable pieces. This reduces code duplication and keeps individual components easy to test, style, and maintain.

9. Routing

React Router DOM defines the application's navigable routes, including the home page (/), authentication routes (/login, /register, /forgot-password), product routes (/products, /product/:id), the shopping routes (/cart, /wishlist, /orders, /profile), and the admin section (/admin, /admin/products, /admin/orders, /admin/users).

10. State Management

Global state is managed with the React Context API rather than an external state-management library. Context stores hold user information, authentication state, the shopping cart, the wishlist, and orders. This approach eliminates prop drilling, centralizes shared state, and keeps the component tree easier to maintain as the application grows.

11. API Integration

Axios is used throughout the application to communicate with the backend REST API. Common requests include login and registration, fetching, adding, updating, and deleting products, fetching and placing orders, and updating user information. Centralizing these calls keeps the UI layer focused on presentation while API logic stays consistent and reusable.

12. Authentication Workflow

On login, the frontend sends the user's credentials to the backend, which verifies them and generates a JWT token. The token is stored on the client and attached to subsequent requests, allowing the backend to authenticate and authorize each call without requiring the user to log in again for every action.

- User submits login credentials.
- Frontend sends the credentials to the backend.
- Backend verifies the user and generates a JWT token.
- Token is stored on the client.
- Subsequent requests are sent as authenticated requests using the stored token.

13. Admin Dashboard

The admin dashboard gives store administrators a single view of the business: statistics cards for totals, sales graphs, and tables for users, products, and orders. Charts are implemented with Recharts, giving administrators

an at-a-glance read on sales and revenue trends alongside the day-to-day product, user, and order management tools.

14. Installation

The application can be run locally with the following steps:

- Clone the repository: `git clone https://github.com/Yug912/E-Commerce-Storefront.git`
- Navigate into the project: `cd E-Commerce-Storefront`
- Install dependencies: `npm install`
- Start the development server: `npm start`
- The application runs at `http://localhost:3000`

15. Major Dependencies

- React
- Material UI
- Axios
- Styled Components
- React Router DOM
- React Toastify
- React Icons
- Recharts

16. Advantages

- Responsive, modern UI across devices.
- Fast, single-page navigation without full reloads.
- Clean REST API integration through Axios.
- Reusable, component-based architecture.
- Secure JWT-based authentication.
- Centralized, prop-drilling-free state management via Context API.
- Interactive admin dashboard with real-time-style statistics and charts.
- Scalable folder structure that supports future feature growth.

17. Current Limitations and Future Enhancements

As a frontend-focused internship project, the application currently assumes a compatible backend API and a configured payment gateway; deeper personalization and offline support are left for future iterations. Planned enhancements include:

- AI-based product recommendations.

- Voice search for product discovery.
- Live chat customer support.
- Dark mode theming.
- A coupon and discount system.
- Multi-language support.
- Product comparison tools.
- Inventory tracking on the admin side.
- Push notifications.
- Progressive Web App (PWA) support for offline access.

18. Learning Outcomes

Building this project provided hands-on experience with:

- React.js fundamentals and component-based design.
- Client-side routing with React Router.
- Global state management using the Context API.
- Consuming REST APIs with Axios.
- Building consistent UIs with Material UI and styled-components.
- Implementing JWT-based authentication flows on the client.
- Responsive, mobile-first design.
- Building and structuring an admin dashboard with data visualization.
- Organizing a scalable, real-world frontend project structure.

19. Resume Summary

Project title: MERN E-Commerce Storefront – Frontend Application

Built a responsive, component-based e-commerce frontend using React.js, Material UI, Axios, React Router DOM, and the Context API, covering customer shopping flows and a full administrator dashboard with data visualization.

- Developed a single-page e-commerce storefront using React.js, Material UI, styled-components, and React Router DOM.
- Implemented JWT-based authentication on the client, including login, registration, and password recovery flows.
- Built cart, wishlist, checkout, and order-history modules with centralized Context API state management.
- Designed and built an administrator dashboard with product, user, and order management, plus Recharts-based sales and revenue visualizations.
- Integrated a REST API using Axios across all customer and admin workflows.

- Structured a scalable folder architecture separating customer, admin, shared components, and context.

20. Conclusion

The MERN E-Commerce Frontend successfully demonstrates the development of a scalable, responsive, and modern client-side web application for an online shopping platform. By leveraging React.js, Material UI, the Context API, Axios, and React Router, the application delivers a smooth user experience while maintaining a modular, maintainable architecture.

The project supports both customer and administrator functionality — authentication, product browsing, cart and wishlist management, secure checkout, order history, and an administrative dashboard with analytics — making it suitable for real-world deployment and further enhancement. It demonstrates practical frontend engineering skill, UI/UX awareness, state management, and REST API integration consistent with production MERN stack development.

References

- [1] React Documentation. [Online]. Available: <https://react.dev/>
- [2] React Router Documentation. [Online]. Available: <https://reactrouter.com/>
- [3] MUI, 'Material UI Documentation.' [Online]. Available: <https://mui.com/>
- [4] Axios Documentation. [Online]. Available: <https://axios-http.com/>
- [5] Recharts Documentation. [Online]. Available: <https://recharts.org/>
- [6] MongoDB, Inc., 'MongoDB Documentation.' [Online]. Available: <https://www.mongodb.com/docs/>
- [7] OpenJS Foundation, 'Express.js Documentation.' [Online]. Available: <https://expressjs.com/>

— *End of Report* —